Razvoj web aplikacija - Predavanje 6
Poslovna pravila i workflow u backendu

mag.ing. Josip Torić (SIT UNIZD)

30.03.2026.

Plan predavanja

1. Što su poslovna pravila i zašto ne žive u routeru
2. Rokovi nominacija: prelim i final deadline
3. Faze natjecanja i što klub smije u kojoj fazi
4. Implementacija u service layeru
5. Domain errori za prekršena pravila
6. Testiranje poslovne logike
7. Priprema za obranu 1. dijela

Na kraju predavanja: backend **odbija** operacije koje krše poslovna pravila, s jasnim error porukama.

Ishodi učenja

Nakon ovog predavanja student može:

- definirati i implementirati vremenski uvjetovana poslovna pravila
- objasniti razliku između validacije inputa i poslovnog pravila
- strukturirati service metodu koja provjerava fazu natjecanja
- napisati test koji verificira ponašanje prije i nakon roka
- koristiti `AppError` za domenske greške s odgovarajućim status codeom

Kontekst: što imamo, što danas zatvaramo

Implementirano (P1-P5)

- Svi modeli i CRUD
- Auth (JWT)
- Autorizacija (role/ownership)
- Validacija i paginacija

Danas

- Rokovi i faze
- Pravila: tko smije što i kad
- Testovi za pravila
- Zaokruživanje za obranu

Poslovna pravila vs validacija inputa

	Validacija inputa	**Poslovno pravilo**
Gdje	Pydantic schema	Service layer
Primjer	`category` mora biti iz IPF liste	klub ne smije prijaviti liftera nakon roka
Status	422	400/403/409
Ovisi o	formatu podataka	stanju sustava

Pydantic provjerava **oblik**. Service provjerava **smisao**.

Faze natjecanja (ponovljeno s P0)

Faza	Što klub smije
Do **prelim** roka	Dodavati nove prijave, uređivati podatke, odjaviti natjecatelja
Između **prelim** i **final**	Odjaviti natjecatelja ili promijeniti kategoriju. **Ne smije** dodavati nove prijave
Nakon **final** roka	Nema izmjena

Ovo su pravila koja backend mora enforcati. Frontend može sakriti gumbe, ali backend **mora odbiti**.

State machine: faze kao konačni automat

Faze natjecanja su **konačni automat** (finite state machine):

$$\text{OPEN} \xrightarrow{\text{prelim_deadline}} \text{PRELIM_PASSED} \xrightarrow{\text{final_deadline}} \text{CLOSED}$$

Svojstva:

- Prijelazi su **jednosmjerni** — ne možemo se vratiti u OPEN
- Prijelaz ovisi o **vremenu**, ne o korisničkoj akciji
- Svaka faza ima jasno definirane **dozvoljene operacije**
- Nema dvosmislenosti — u svakom trenutku faza je točno jedna

Kad pravila postanu složenija (npr. 5+ stanja, korisničke tranzicije), razmislite o biblioteci za state machine.

Soft delete vs hard delete

	Hard delete (`DELETE`)	**Soft delete** (status)
Što se dogodi	Red nestaje iz baze	Red ostaje, `status` se mijenja
Audit trail	Nema (osim logova)	Potpun — vidimo tko je i kad odjavljen
Undo	Nemoguće	Moguće (vrati status na `"active"`)
Query složenost	Jednostavni	Treba filtrirati po statusu

Naš projekt koristi soft delete za registracije (`status = "withdrawn"`) jer:

- Organizator treba znati da je netko bio prijavljen pa se odjavio
- Sprječava ponovnu prijavu istog liftera (unique constraint i dalje vrijedi)

Idempotentnost: što ako korisnik klikne dva puta?

Operacija je **idempotentna** ako ponovljeni poziv daje isti rezultat.

Operacija	Idempotentna?	Naša implementacija
GET lista	Da	Uvijek isti rezultat
POST registracija	Ne	Drugi poziv: 409 (duplikat)
POST withdraw	Ne	Drugi poziv: 400 (`already_withdrawn`)
PATCH kategorija	Da	Isti rezultat koliko god puta

Za ne-idempotentne operacije, frontend treba spriječiti duple klikove (disable button dok traje request). Backend ih odbija s jasnim error codeom — frontend zna da nije greška, nego dupli poziv.

Određivanje faze: pomoćna funkcija

Fazu određujemo usporedbom trenutnog vremena s rokovima natjecanja.

Tri moguća rezultata:

1. `OPEN` - prije prelim roka
2. `PRELIM_PASSED` - između prelim i final roka
3. `CLOSED` - nakon final roka

Ova funkcija ne ovisi o korisniku ni o requestu - samo o natjecanju i satu.

Implementacija: faza natjecanja

```
from enum import Enum
from datetime import datetime, timezone

class Phase(str, Enum):
    OPEN = "open"
    PRELIM_PASSED = "prelim_passed"
    CLOSED = "closed"

def get_competition_phase(competition: Competition) -> Phase:
    now = datetime.now(timezone.utc)
    if now < competition.prelim_deadline:
        return Phase.OPEN
    if now < competition.final_deadline:
        return Phase.PRELIM_PASSED
    return Phase.CLOSED
```

Service: registracija s provjerom faze

```
async def create_registration(
    db: AsyncSession,
    comp_id: int,
    body: RegistrationCreate,
    current_user: User,
) -> Registration:
    competition = await competition_repo.get_by_id(db, comp_id)
    if not competition:
        raise AppError("not_found", "Natjecanje ne postoji", 404)

    phase = get_competition_phase(competition)
    if phase != Phase.OPEN:
        raise AppError(
            "deadline_passed",
            "Rok za nove prijave je istekao",
            400,
        )

    lifter = await lifter_repo.get_by_id(db, body.lifter_id)
    if current_user.role == "club":
        if lifter.club_id != current_user.club_id:
            raise AppError("forbidden", "Pristup odbijen", 403)

    reg = Registration(
        competition_id=comp_id, **body.model_dump()
    )
    return await registration_repo.create(db, reg)
```

Service: odjava natjecatelja (soft status)

```
async def withdraw_registration(
    db: AsyncSession,
    comp_id: int,
    registration_id: int,
    current_user: User,
) -> Registration:
    reg = await registration_repo.get_by_id(db, registration_id)
    if not reg or reg.competition_id != comp_id:
        raise AppError("not_found", "Prijava ne postoji", 404)
    if reg.status == "withdrawn":
        raise AppError("already_withdrawn", "Vec odjavljen", 400)

    # ownership
    lifter = await lifter_repo.get_by_id(db, reg.lifter_id)
    if current_user.role == "club":
        if lifter.club_id != current_user.club_id:
            raise AppError("forbidden", "Pristup odbijen", 403)

    phase = get_competition_phase(
        await competition_repo.get_by_id(db, comp_id)
    )
    if phase == Phase.CLOSED:
        raise AppError(
            "deadline_passed",
            "Izmjene nisu moguce nakon finalnog roka",
            400,
        )

    reg.status = "withdrawn"
    await db.flush()
    return reg
```

Soft status: ne brišemo red, mijenjamo `status` u `"withdrawn"`. Čuvamo audit trail.

Matrica operacija po fazama

Operacija	OPEN	PRELIM	CLOSED	Error
Nova prijava	da	ne	ne	400
Promjena kategorije	da	da	ne	400
Odjava natjecatelja	da	da	ne	400
Pregled prijava	da	da	da	-

Svaka operacija u serviceu mora provjeriti fazu prije izvršenja.

Promjena kategorije: OPEN i PRELIM_PASSED

```
async def update_registration_category(
    db: AsyncSession,
    comp_id: int,
    registration_id: int,
    new_category: str,
    current_user: User,
) -> Registration:
    reg = await registration_repo.get_by_id(db, registration_id)
    if not reg or reg.competition_id != comp_id:
        raise AppError("not_found", "Prijava ne postoji", 404)

    # ownership
    lifter = await lifter_repo.get_by_id(db, reg.lifter_id)
    if current_user.role == "club":
        if lifter.club_id != current_user.club_id:
            raise AppError("forbidden", "Pristup odbijen", 403)

    phase = get_competition_phase(
        await competition_repo.get_by_id(db, comp_id)
    )
    if phase == Phase.CLOSED:
        raise AppError(
            "deadline_passed",
            "Izmjene nisu moguce nakon finalnog roka",
            400,
        )

    reg.category = new_category
    await db.flush()
    return reg
```

Hvatanje IntegrityError: dupla prijava

Unique constraint na `(lifter_id, competition_id)` baca `IntegrityError` pri duplu pokušaju.

Dva pristupa:

1. Provjeri u kodu prije inserta (query + if) - ne štiti od race conditiona
2. Hvati `IntegrityError` nakon flusha - sigurno, baza je autoritet

Koristimo oba: query daje razumljivu grešku, constraint je sigurnosna mreža.

Hvatanje IntegrityError u serviceu

```
from sqlalchemy.exc import IntegrityError

async def create_registration(db, comp_id, body, current_user):
    # ... faza i ownership provjere ...

    reg = Registration(competition_id=comp_id, **body.model_dump())
    try:
        return await registration_repo.create(db, reg)
    except IntegrityError:
        raise AppError(
            "duplicate_registration",
            "Natjecatelj je vec prijavljen na ovo natjecanje",
            409,
        )
```

409 Conflict - klijent zna da je resurs vec postoji i može prilagoditi UI.

Domain errori: dosljedan format

Svaki error ima tri dijela:

Polje	Tip	Primjer
`code`	string	`deadline_passed`
`message`	string	„Rok za nove prijave je istekao”
`status_code`	int	400

`code` je strojno čitljiv - frontend može reagirati na njega (`switch` po codeu).
`message` je za čovjeka.

Pregled svih error codeova u projektu

Code	HTTP	Kad
`invalid_credentials`	401	login s krivom lozinkom
`token_expired`	401	istekao JWT
`forbidden`	403	nema pristup (rola/ownership)
`not_found`	404	resurs ne postoji
`duplicate_registration`	409	dupla prijava
`already_withdrawn`	400	ponovna odjava vec odjavljenog
`deadline_passed`	400	operacija izvan dozvoljene faze
`validation_error`	422	krivi format inputa

Testiranje poslovnih pravila: strategija

Tri tipa testova za pravila:

1. **Happy path**: operacija uspije kad su uvjeti zadovoljeni
2. **Prekršaj pravila**: operacija je odbijena s točnim error codeom
3. **Granični slučaj**: operacija na samoj granici roka

Za rokove je ključno kontrolirati „sada” u testovima - ne želimo da testovi ovise o stvarnom satu.

Freezegun: kontrola vremena u testovima

```
from freezegun import freeze_time

@freeze_time("2026-03-01 10:00:00")
def test_registration_before_prelim(client, admin_token):
    # prelim_deadline je 2026-03-15
    r = client.post(
        "/competitions/1/registrations",
        json={"lifter_id": 1, "category": "83"},
        headers={"Authorization": f"Bearer {admin_token}"},
    )
    assert r.status_code == 201

@freeze_time("2026-03-20 10:00:00")
def test_registration_after_prelim(client, admin_token):
    r = client.post(
        "/competitions/1/registrations",
        json={"lifter_id": 1, "category": "83"},
        headers={"Authorization": f"Bearer {admin_token}"},
    )
    assert r.status_code == 400
    assert r.json()["code"] == "deadline_passed"
```

Alternativa freezegunu: injektiranje vremena

- Umjesto `datetime.now()` u kodu, primamo `now` kao parametar
- Service metoda: `def create_registration(db, body, user, now=None)`
- U produkciji: `now` je `None` → koristi se `datetime.now(utc)`
- U testu: šaljemo fiksnu vrijednost

Oba pristupa su validna. Freezegun je praktičniji za integration testove, injektiranje za unit testove.

Test: odjava prije i nakon final roka

```
@freeze_time("2026-03-20 10:00:00")
def test_withdraw_between_prelim_and_final(
    client, club_token, registration_id
):
    # final_deadline je 2026-04-01
    r = client.post(
        f"/competitions/1/registrations/{registration_id}/withdraw",
        headers={"Authorization": f"Bearer {club_token}"},
    )
    assert r.status_code == 200
    assert r.json()["status"] == "withdrawn"

@freeze_time("2026-04-05 10:00:00")
def test_withdraw_after_final(
    client, club_token, registration_id
):
    r = client.post(
        f"/competitions/1/registrations/{registration_id}/withdraw",
        headers={"Authorization": f"Bearer {club_token}"},
    )
    assert r.status_code == 400
    assert r.json()["code"] == "deadline_passed"
```

Test: dupla prijava daje 409

```
@freeze_time("2026-03-01 10:00:00")
def test_duplicate_registration(client, admin_token):
    payload = {"lifter_id": 1, "category": "83"}
    headers = {"Authorization": f"Bearer {admin_token}"}
    url = "/competitions/1/registrations"

    r1 = client.post(url, json=payload, headers=headers)
    assert r1.status_code == 201

    r2 = client.post(url, json=payload, headers=headers)
    assert r2.status_code == 409
    assert r2.json()["code"] == "duplicate_registration"
```

Što pokrivamo testovima prije obrane

1. Auth: login uspješan, login neuspješan, pristup bez tokena
2. Role: admin smije, club ne smije (i obratno)
3. Ownership: club A ne smije pristupiti podacima kluba B
4. Rokovi: registracija prije/između/nakon rokova
5. Constraint: dupla prijava daje 409
6. Validacija: krivi input daje 422

Na obrani mogu pokrenuti testove i pitati **što točno svaki test pokriva**.

Commitovi za Predavanje 6

- `feat(api): add Phase enum and get_competition_phase helper`
- `feat(api): enforce phase rules on registration create and category update`
- `feat(api): add withdraw endpoint with status transition and phase check`
- `test(api): add phase-based tests with freezegun (OPEN/PRELIM/CLOSED)`
- `test(api): add withdrawal and edge case tests`

Na satu: prolazimo diffeve, pokrećemo testove, simuliramo scenarije u Swaggeru.

Rekapitulacija: cijeli backend stack

1. **P1** - Skeleton, zdravlje, konfiguracija
2. **P2** - Modeli, ORM, migracije, seed
3. **P3** - Autentikacija (JWT, bcrypt, login/refresh)
4. **P4** - Autorizacija (role, ownership, 401/403)
5. **P5** - CRUD, schemed, validacija, paginacija
6. **P6** - Poslovna pravila, rokovi, workflow

Sve zajedno: korisnik se prijavi, ima rolu, vidi samo svoje, poštuje rokove, dobiva jasne greške.

Priprema za obranu (13.04.)

1. Pokreni projekt: `docker compose up -d db` + `uvicorn`
2. Pokaži CRUD u Swaggeru
3. Pokreni `pytest` - svi testovi prolaze
4. Budi spreman objasniti:
   - Gdje provjeravaš ownership?
   - Kako backend zabranjuje izmjene nakon roka?
   - Zašto je status code baš taj (401/403/409)?
   - Što test pokriva i zašto je bitan?

Mini promjena na obrani

Na obrani mogu tražiti malu promjenu uživo:

- Dodaj novo polje u model + migracija
- Dodaj constraint ili provjeru u service
- Promijeni error poruku ili status code
- Dodaj novi test za edge case

Nema potrebe za panikom. Ako razumijete slojeve i znate gdje što živi, promjena traje par minuta.

Što ide sljedeće (Predavanje 7)

- Arhitektura frontend aplikacija
- Vue 3: komponente, reaktivnost, Composition API
- Vite dev server i build pipeline
- Struktura SPA projekta

Pitanja?